Geavanceerde Computerarchitectuur Labo 1

Arno De Lathauwer
Indra Coppens

February 2025

1 Context

Tijdens dit labo is het de bedoeling om de maximumwaarde te bepalen in een array. Dit doen we op drie verschillende manieren. Eerst implementeren we de oplossing met een for-lus op de CPU. Vervolgens passen we een GPU-kernel toe met atomic operations. Ten slotte maken we gebruik van reduction met een GPU-kernel. Daarnaast onderzoeken we hoe de verwerkingstijd toeneemt naarmate de grootte van de input groeit.

2 Methodologie

Als eerste maken we een programma in `C++` dat de hoogste waarde in een array zoekt, gebruik makend van een for-loop. Ieder element van de array wordt afgelopen en vergeleken met de hoogste waarde die tot dan toe gevonden is. Deze waarde zit opgeslagen in de variabele `highNum`. Wanneer het huidige element groter is dan de tot dan toe hoogste waarde (`list[m]` $>$ `highNum`), wordt de hoogste waarde gelijk gesteld aan het huidige element (`highNum = list[m]`). Nadat de array volledig doorlopen is zal de variabele `highNum` de hoogste waarde in de array bevatten. Dit is te zien in Appendix A.

Vervolgens wordt een GPU-kernel functie geschreven die opnieuw de hoogste waarde in een array zoekt. Specifiek wordt hier gebruik gemaakt van de atomic operation `atomicMax()`. Op de CPU wordt een array met waarden in (`h_arr`) aangemaakt. Deze wordt vervolgens, samen met de integer `h_maxVal` naar het geheugen van de GPU gekopieerd. De kernel wordt uitgevoerd met één blok met evenveel threads als elementen in de array: iedere thread bepaalt het maximum van `maxVal` en `d_arr[idx]` en slaat het maximum op in `maxVal` [1]. De atomic operation voorkomt race condities waardoor de waarde van `maxVal` correct wordt aangepast. De code voor deze functie is te zien in Appendix B.

In een tweede GPU-kernel functie wordt gebruik gemaakt van reduction om de hoogste waarde in de array te zoeken. De kernel wordt uitgevoerd met één block met `N/2` threads (waarbij `N` het aantal elementen in de array is). Er worden $\log_2$(`size`) iteraties uitvoerd (hierbij is `size` gelijk aan het aantal elementen in de array). Voor een array met 16 elementen hebben we dus vier iteraties nodig, zie Figuur 1. Vervolgens wordt voor iedere thread bepaalt welke twee elementen vergeleken zullen worden:

Index element 1: `int val1 = idx * pow (2 , iteratie ) ;`
Index element 2: `int val2 = idx * pow (2 , iteratie ) + ( pow (2 , iteratie ) /2) ;`

Deze elementen worden met elkaar vergeleken, en het grootste element wordt op de laagste van de twee indexen in de array geplaatst. Na het doorlopen van alle iteraties, zal het element met de grootste waarde vooraan in de array staan. Dit is te zien in Appendix C.

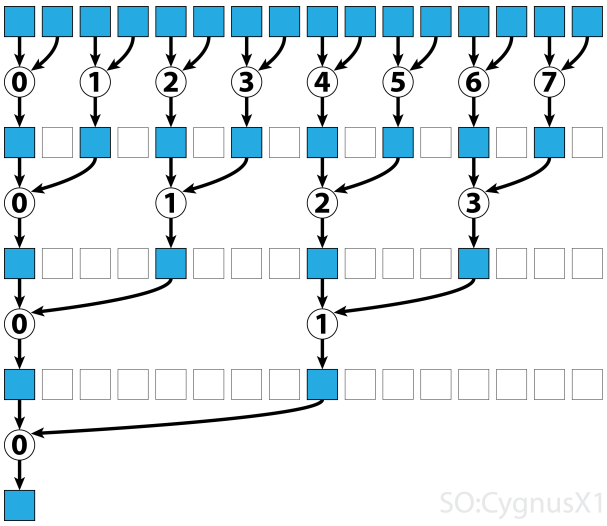


Figure 1: Reduction [3]

In bovenstaande GPU-kernel functies werd er telkens met één blok gewerkt. Wanneer we gebruik willen maken van meerdere blokken, zullen er problemen optreden (in de bovenstaande GPU-kernel functies) omwille van de `__syncthreads`-functie die enkel werkt voor threads die deel uitmaken van éénzelfde blok. Dit probleem werd opgelost door te itereren op de CPU. Dit houdt in dat de parallelle functies uitgevoerd worden op de GPU en hierna telkens gesynchroniseerd wordt op de CPU alvorens verder te gaan naar de volgende iteratie. Op die manier worden de threads over alle blokken gesynchroniseerd bij elke iteratie. Deze aanpassing is te zien in Appendix D.

3 Observaties/resultaten

3.1 Correctness of program

In alle vier de uitwerkingen:

1. For-loop op CPU
2. GPU-kernel met atomic operations
3. GPU-kernel met reduction (één block)
4. GPU-kernel met reduction (meerdere blocks)

geeft het programma een correcte output. Alle programma's slagen er dus in om de grootste waarde in een array te vinden.

3.2 Time increase as a function of input size

Elke functie wordt gerund met telkens 10 verschillende array-groottes (2, 4, 8, 16, 32, 64, 128, 256, 512 en 1024 elementen) en dit 500 keer (voor een totaal van 5000 runs). Deze data wordt geëxporteerd naar een output file en vervolgens ingelezen in een Python programma die de execution time uitzet in functie van de array size in een boxplot. Deze boxplots zijn te vinden in figuren 2, 3, 4 en 5.

Voor alle uitwerkingen zien we dat de execution time toeneemt naarmate het aantal elementen in de array toeneemt. We zien tevens ook dat de execution time voor het programma op de CPU sterker toeneemt voor grotere array sizes dan bij programma's die gerund worden op de GPU. Dit is logisch aangezien de programma's op de GPU gebruik maken van parallellisatie. Waardoor de verschillende instructies tegelijkertijd in plaats van sequentieel uitgevoerd kunnen worden. Daarnaast is het ook logisch dat voor arrays met een grotere size de GPU programma's ook trager zijn. Er moeten meer iteraties uitgevoerd worden (in het geval van reduction) en er is meer resource contention (in het geval van atomic operations).

3.3 Big input (> 2048)

In het geval van de implementatie via reductie kan voor een array met een input size van 2048 elementen nog de maximale waarde gevonden worden. Wanneer we echter naar een array gaan met een size groter dan 2048 elementen, zal de GPU-kernel functie niet uitgevoerd worden en zal de maximale waarde niet gezocht worden in de array. Dit komt doordat het aantal threads per block beperkt is op 1024. Door met meerdere blokken te werken is het wel mogelijk om arrays met een size groter dan 2048 te verwerken, aangezien we dan (in totaal) wel meer dan 1024 threads kunnen aanspreken.

3.4 Using much less threads

Wanneer we veel minder threads ter beschikking hebben (per block) dan nodig zijn, zijn er twee manieren om toch te garanderen dat de maximale waarde in de array wordt gevonden. Deze manieren zijn:

1. Meerdere blokken gebruiken, waardoor in totaal toch een groot genoeg aantal threads beschikbaar is.

2. Gebruik maken van striding. Dit houdt in dat een thread hergebruikt wordt om meerdere elementen te berekenen (door middel van een for loop in de GPU-kernel functie) [2]. In plaats van met iedere thread telkens één element te berekenen.

3.5 2D array

De grootste waarde in een 2D array zou gezocht kunnen worden door in iedere rij van de matrix de grootste waarde te zoeken en deze vervolgens toe te voegen aan een aparte array of vooraan de rij te zetten. Daarna kan in deze aparte array of in de 1ste kolom van de matrix (waar nu de grootste waarde van iedere rij staat) opnieuw de grootste waarde gezocht worden, wat de grootste waarde in de 2D array oplevert.

4 Conclusie

In dit labo hebben we geleerd hoe we gebruik kunnen maken van atomic operations en/of reduction om bewerkingen op een array in parallel uit te voeren. Daarnaast hebben we aangetoond dat voor arrays met een grote(re) size de execution time toeneemt.

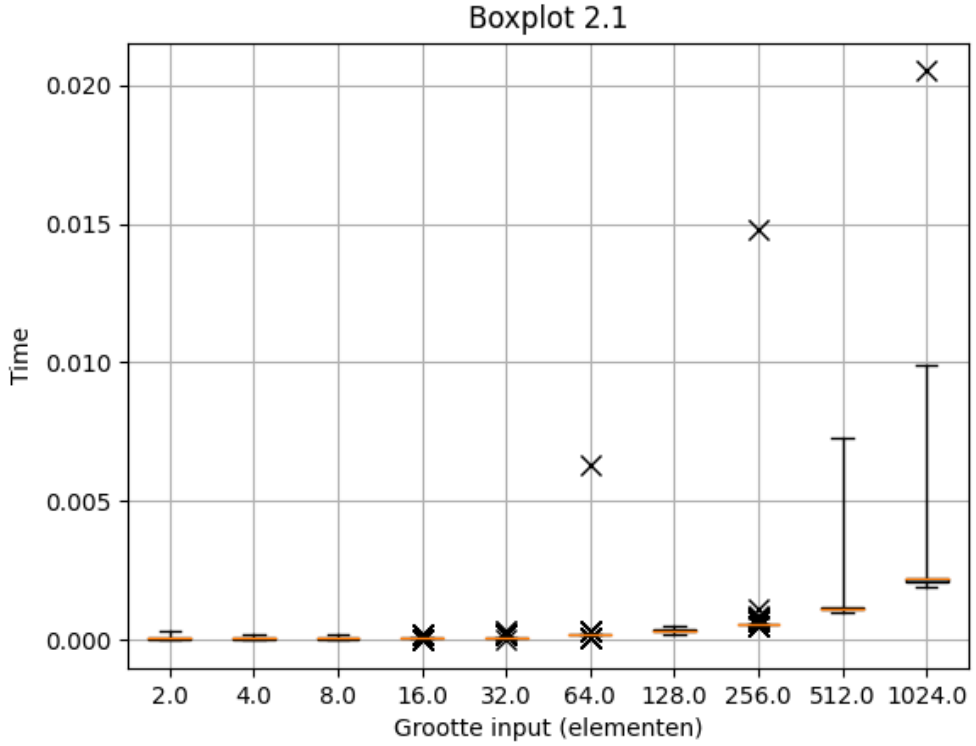


Figure 2: Time in function of input size with for-loop on the CPU

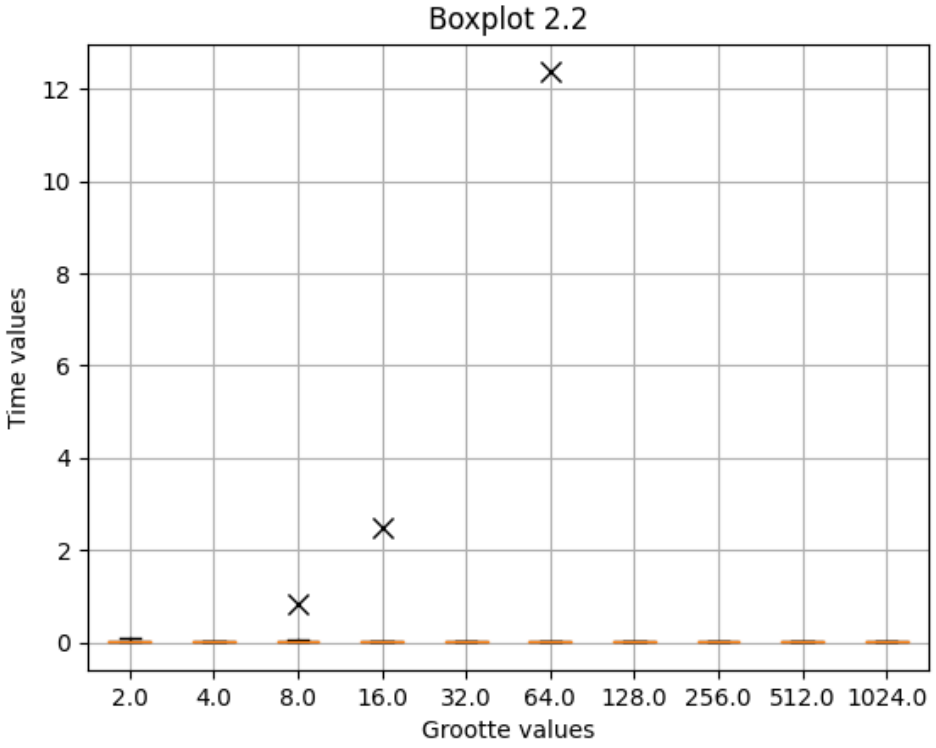


Figure 3: Time in function of input size for a CPU kernel employing atomic operations

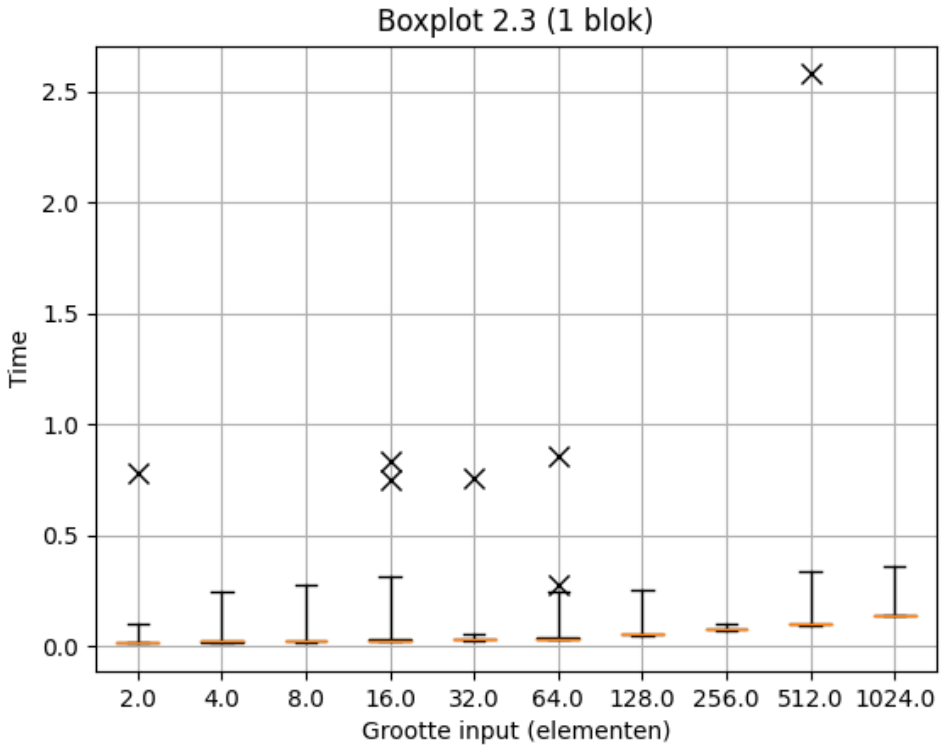


Figure 4: Time in function of input size for GPU kernel using reduction (one block)

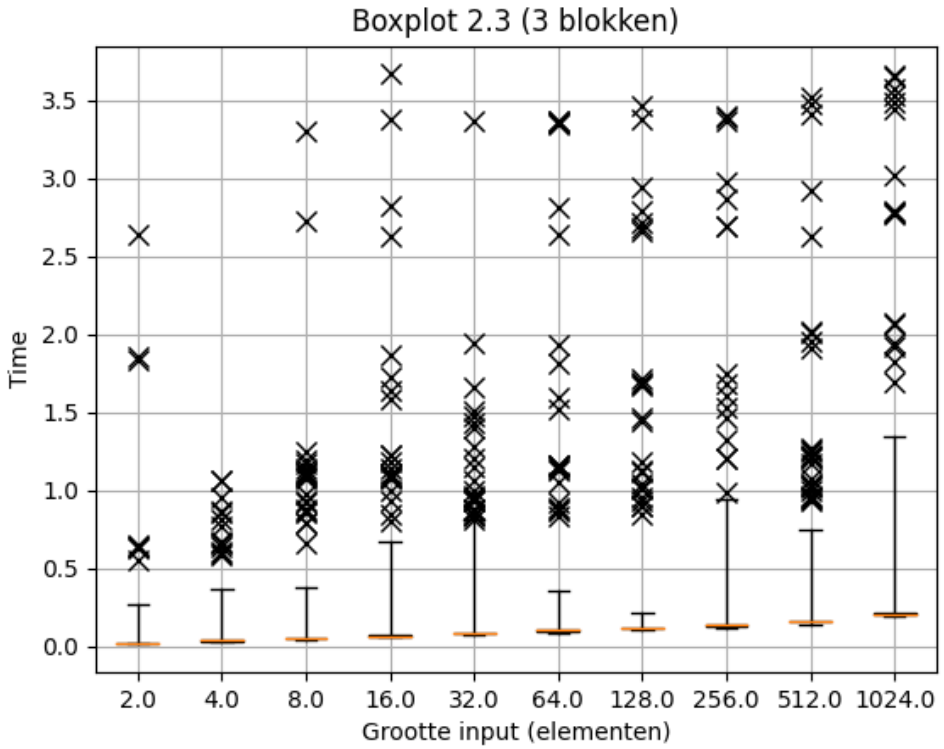


Figure 5: Time in function of input size for GPU kernel using reduction (3 blocks)

Appendix A: Code: for-loop on the CPU

```cpp
#include <chrono>
#include <fstream>
#include <iostream>
#include <vector>
using namespace std;

int main() {
  vector<float> time;
  vector<int> size;
  for (int j = 0; j < 500; j++) {
    for (int i = 2; i < 1025; i = i * 2) {
      int N = i;
      size.push_back(N);
      int list[N];
      for (int e = 0; e < N; e++) {
        list[e] = e;
      }
      int highNum = 0;
      // int size = 4;
      // int list[size] = {10, 4, 7, 8};

      int m;
      auto start = chrono::high_resolution_clock::now();
      for (m = 0; m < N; m++) {
        if (list[m] > highNum)
          highNum = list[m];
      }
      auto end = chrono::high_resolution_clock::now();
      chrono::duration<double, std::milli> duration = end - start;
      float milliseconds = duration.count();
      cout << "Highest number is " << highNum << ", duration: " << milliseconds
           << " ms"
           << "\n";
      time.push_back(milliseconds);
    }
  }

  // Schrijf naar een bestand
  ofstream outFile("lab2_1_time.txt");
  if (!outFile) {
    cerr << "Kon bestand niet openen!" << endl;
    return 1;
  }

  for (size_t i = 0; i < time.size(); i++) {
    outFile << time[i] << " " << size[i] << "\n";
  }

  outFile.close();
  cout << "Data opgeslagen in data.txt\n";
  return 0;
}
```

Appendix B: Code: GPU kernel employing atomic operations

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <cuda_runtime.h>


__global__ void maxArray(int *d_arr, int *maxVal, int size) {
    int idx = threadIdx.x;// + blockIdx.x * blockDim.x;
    if(idx < size){
        atomicMax(maxVal, d_arr[idx]);
    }
}
```

```
13
14  int main() {
15      std::vector<float> time;
16      std::vector<int> size;
17      for(int j = 0; j<500; j++){
18          for(int i = 2; i<1025;i=i*2){
19              int N = i;
20              size.push_back(N);
21              int h_arr[N];
22              for(int e = 0; e<N; e++){
23                  h_arr[e] = e;
24              }
25              //int h_arr[N] = {1,2,3,4,5,6,21,8,9,10}; // Host-array (CPU)
26              int h_maxVal =0;
27
28              int *d_arr; // Device-array (GPU)
29              int* d_maxVal;
30
31                  cudaEvent_t start, stop;
32                  cudaEventCreate(&start);
33                  cudaEventCreate(&stop);
34                  cudaMalloc((void**)&d_arr, N * sizeof(int));
35                  cudaMalloc((void**)&d_maxVal, sizeof(int));
36                  cudaMemcpy(d_arr, h_arr, N * sizeof(int), cudaMemcpyHostToDevice);
37                  cudaMemcpy(d_maxVal, &h_maxVal, sizeof(int), cudaMemcpyHostToDevice);
38                  cudaEventRecord(start);
39                  maxArray<<<1, N >>>(d_arr, d_maxVal, N);
40                  cudaEventRecord(stop);
41                  cudaDeviceSynchronize();
42
43                  cudaMemcpy(&h_maxVal, d_maxVal, sizeof(int), cudaMemcpyDeviceToHost);
44                  cudaEventSynchronize(stop);
45
46                  float milliseconds = 0;
47                  cudaEventElapsedTime(&milliseconds, start, stop);
48                  time.push_back(milliseconds);
49
50                  std::cout << "array: ";
51                  for(i = 0; i< N; i++){
52                      std::cout << h_arr[i] << " ";
53                  }
54                  std::cout <<  " \n";
55                  std::cout << "Max waarde: " << h_maxVal << " \n";
56                  std::cout << "Tijd nodig: " << milliseconds << " milliseconden - grootte
                        : " << i << "\n";
57
58
59
60                  // Opruimen
61                  cudaFree(d_arr);
62                  cudaFree(d_maxVal);
63
64
65          }
66
67      }
68      // Schrijf naar een bestand
69          std::ofstream outFile("lab2_2_time.txt");
70          if (!outFile) {
71              std::cerr << "Kon bestand niet openen!" << std::endl;
72              return 1;
73          }
74
75          for (size_t i = 0; i < time.size(); i++) {
76              outFile << time[i] << " " << size[i] << "\n";
77          }
78
79          outFile.close();
80          std::cout << "Data opgeslagen in lab2_2_time.txt\n";
81      return 0;
82  }
```

Appendix C: Code: GPU kernel using reduction with timer and one block

```
#include <iostream>
#include <fstream>
#include <vector>
#include <cuda_runtime.h>


__global__ void maxArray(int *d_arr, int log_2_size) {

    for(int iteratie = 1; iteratie <= log_2_size; iteratie++){
        int idx = threadIdx.x + blockIdx.x * blockDim.x;
        //enkel threads die nodig zijn gebruiken voor die iteratie
        //check hoeft niet pers : want zit buiten array en er worden toch telkens 32
            threads gescheduled onderliggend
        if(idx < pow(2, log_2_size)/(pow(2,iteratie))){
            int val1 = idx*pow(2,iteratie) ;
            int val2 = idx*pow(2,iteratie) + (pow(2,iteratie)/2);

            if(d_arr[val2] > d_arr[val1]){
                d_arr[val1] = d_arr[val2];
            }
            __syncthreads();
        }

    }
}

int main() {
    std::vector<float> time;
    std::vector<int> size;
    for(int j = 0; j<500; j++){
    for(int i = 2; i<1025;i=i*2){
        int N = i;
        size.push_back(N);
        int h_arr[N];
        for(int e = 0; e<N; e++){
            h_arr[e] = e;
        }
        int *d_arr; // Device-array (GPU)

        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaMalloc((void**)&d_arr, N * sizeof(int));
        cudaMemcpy(d_arr, h_arr, N * sizeof(int), cudaMemcpyHostToDevice);

        cudaEventRecord(start);
        maxArray<<<1, N/2 >>>(d_arr, log2(N));
        cudaEventRecord(stop);
        cudaDeviceSynchronize();

        cudaMemcpy(h_arr, d_arr, N*sizeof(int), cudaMemcpyDeviceToHost);

        cudaEventSynchronize(stop);
        float milliseconds = 0;
        cudaEventElapsedTime(&milliseconds, start, stop);
        time.push_back(milliseconds);

        std::cout << "array: ";
        for(i = 0; i< N; i++){
            std::cout << h_arr[i] << " ";
        }
        std::cout <<  " \n";
        std::cout << "Max waarde: " << h_arr[0] << " \n";
        std::cout << "Tijd nodig: " << milliseconds << " milliseconden - grootte: " << i
             << "\n";


        // Opruimen
```

```
        cudaFree(d_arr);

    }
    }

    // Schrijf naar een bestand
    std::ofstream outFile("lab2_3_time.txt");
    if (!outFile) {
        std::cerr << "Kon bestand niet openen!" << std::endl;
        return 1;
    }

    for (size_t i = 0; i < time.size(); i++) {
        outFile << time[i] << " " << size[i] << "\n";
    }

    outFile.close();
    std::cout << "Data opgeslagen in data.txt\n";

return 0;
}
```

Appendix D: Code: GPU kernel using reduction with timer and multiple blocks

```
#include <iostream>
#include <fstream>
#include <vector>
#include <cuda_runtime.h>


__global__ void maxArray(int *d_arr, int log_2_size, int iteratie) {

        int idx = threadIdx.x + blockIdx.x * blockDim.x;
        //enkel threads die nodig zijn gebruiken voor die iteratie
        //check hoeft niet: want zit buiten array en er worden toch telkens 32 threads
            gescheduled onderliggend
        if(idx < pow(2, log_2_size)/(pow(2,iteratie))){
            int val1 = idx*pow(2,iteratie) ;
            int val2 = idx*pow(2,iteratie) + (pow(2,iteratie)/2);

            if(d_arr[val2] > d_arr[val1]){
                d_arr[val1] = d_arr[val2];
            }
            __syncthreads();
        }
}

int main() {
    std::vector<float> time;
    std::vector<int> size;
    for(int j = 0; j<500; j++){
        for(int i = 2; i<1025;i=i*2){
            int N = i;
            size.push_back(N);
            int h_arr[N];
            for(int e = 0; e<N; e++){
                h_arr[e] = e;
            }
            int *d_arr; // Device-array (GPU)
            int iteratie;
                cudaEvent_t start, stop;
            cudaEventCreate(&start);
            cudaEventCreate(&stop);

            cudaMalloc((void**)&d_arr, N * sizeof(int));
            cudaMemcpy(d_arr, h_arr, N * sizeof(int), cudaMemcpyHostToDevice);

            cudaEventRecord(start);
```

```
            for(iteratie = 1; iteratie <= log2(N); iteratie ++){
                maxArray<<<3, N/2/3+1 >>>(d_arr, log2(N), iteratie);
                cudaDeviceSynchronize();
            }
            cudaEventRecord(stop);

            cudaMemcpy(h_arr, d_arr, N*sizeof(int), cudaMemcpyDeviceToHost);

            cudaEventSynchronize(stop);
            float milliseconds = 0;
            cudaEventElapsedTime(&milliseconds, start, stop);
            time.push_back(milliseconds);

            std::cout << "array: ";
            for(i = 0; i< N; i++){
                std::cout << h_arr[i] << " ";
            }
            std::cout <<  " \n";
            std::cout << "Max waarde: " << h_arr[0] << " \n";
            std::cout << "Tijd nodig: " << milliseconds << " milliseconden - grootte: "
                << i << "\n";

            // Opruimen
            cudaFree(d_arr);

        }
    }
    // Schrijf naar een bestand
    std::ofstream outFile("data.txt");
    if (!outFile) {
        std::cerr << "Kon bestand niet openen!" << std::endl;
        return 1;
    }

    for (size_t i = 0; i < time.size(); i++) {
        outFile << time[i] << " " << size[i] << "\n";
    }

    outFile.close();
    std::cout << "Data opgeslagen in data.txt\n";

return 0;
}
```

References

[1] 'CUDA C++ Programming Guide'. Accessed: Feb. 18, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[2] 'CUDA Grid-Stride Loops: What if you Have More Data Than Threads?'. Accessed: Feb. 22, 2025. [Online]. Available: https://alexminnaar.com/2019/08/02/grid-stride-loops.html

[3] 'cuda Tutorial => Single-block parallel reduction for...' Accessed: Feb. 22, 2025. [Online]. Available: https://riptutorial.com/cuda/example/22457/single-block-parallel-reduction-for-non-commutative-operator